

Where Does the Graph Help? Seed Expansion for Function-Level Bug Localization on Repository Code Graphs

Vitor Calvi · Kaggle: Google — The Gemma 4 Developer Agent Paper Track · 30 September 2026 Target award: Best New Resource (\$10,000) / Overall Best Paper (\$15,000)

Abstract

Code-structure artifacts — call graphs, import graphs, node embeddings — are now standard equipment for repository-level coding agents, and the Gemma 4 Developer Agent competition ships both. What is missing is a measurement of *what they actually buy*. We build a function-level localization benchmark over all 300 SWE-bench Lite instances using the competition’s own codegraph/v1 graphs, and answer three questions with an exact, CPU-only, LLM-free protocol. (1) The task is single-target: 80.7% of patches edit exactly one function in exactly one file, and the target is present in the provided graph 100% of the time, so coverage is not the bottleneck — ranking is. (2) Lexical retrieval has a catastrophic tail: the median gold function sits at rank 32, but 39.0% of tasks rank it outside the top 100 and 10.7% outside the top 5 000. We identify the mechanism as **public-API / private-implementation divergence** — issues name the symptom surface while patches edit a helper one or two call hops away. (3) Expanding the lexical seed set over the call graph raises coverage@50 from 0.379 to 0.562 with **zero** tasks lost (exact McNemar $p = 7.3e-12$) — but a controlled ablation shows this gain is a *bigger seed set*, not smarter ordering: at a fixed budget of 20–50 candidates, graph-aware ordering is **no better than plain BM25** (-0.014 , $p = 0.61$), and the only settings that dominate the lexical baseline at tight budgets use a **small** seed set ($k=5$ reaches coverage@10 0.410 vs 0.379, while $k=1$ collapses to 0.276). Crucially, we also report five negative results: a learned reranker over 12 graph and lexical features does **not** beat plain BM25 ordering ($p = 1.0$), seed-consensus ranking is actively harmful, and personalized PageRank fusion adds ≈ 1 point. Graphs decide *where to look*; nothing we tried decides *what to look at first*. We release the benchmark labels, the library, and a public notebook that regenerates every number in under three minutes.

1. Introduction

A coding agent given a bug report must answer one question before it can write a patch: *which function do I edit?* Modern repository-level agents answer it with graph structure. The Gemma 4 Developer Agent competition exposes `get_code_neighbors`, `search_similar_code`, and `get_code_subgraph` over a per-repository code graph, and its organizers explicitly invite research on “code-graph generation, parsing, and embedding” and “graph reasoning”. Meanwhile the localization literature reports $\text{accuracy}@k$ for retrieval methods on SWE-bench without ever publishing the *per-instance geometry* those numbers summarize.

That omission is costly. Two papers can report “ $\text{acc}@10 = 0.45$ ” on the same benchmark with no way to tell whether one is easier, whether both are near a ceiling, or where the remaining failures live. We fill that gap. Our contribution is a **reusable, fully specified difficulty benchmark** for function-level localization on repository code graphs, plus the measurements it makes possible.

We make three claims, each measured on all 300 instances:

- **C1 (the task shape).** Localization here is single-target search, not multi-site editing. 80.7% of reference patches modify exactly one function in exactly one file, and that function is present in the provided graph essentially always. Graph *coverage* is solved; graph *ranking* is not.
- **C2 (the mechanism).** Lexical retrieval fails on a third of tasks because of a systematic vocabulary mismatch we call public-API / private-implementation divergence: the issue names a public symbol, the patch edits a private helper it calls. In our worked example the issue is written entirely about `separability_matrix`, which BM25 ranks **first**, while the patch edits `_cstack`, which BM25 ranks **5,758** of 8,452, three call hops away in the same file.
- **C3 (the fix, and its limit).** One to three hops of explicit expansion over the call graph converts that mismatch into recall — $\text{coverage}@50$ rises $0.379 \rightarrow 0.562$ with zero regressions. But *ordering* the expanded pool is unsolved: five ranking strategies, including a learned one, all fail to beat plain BM25.

We report C3’s limit as prominently as its gain.

2. Related work

Repository-level retrieval. RepoCoder (Zhang et al., 2023) frames repository context as an iterative retrieve-and-generate loop and shows structure-aware chunking over AST nodes beats fixed windows. RepoFuse (Shrivastava et al., 2023) fuses lexical similarity with repository-level data-flow into a single context vector. CodeRAG-Bench (Wang et al., 2025) is the closest diagnostic: it sweeps retrieval sources and reports that “current retrievers still struggle to fetch useful contexts especially with limited lexical overlap.” Our C2 is a precise, instance-level instance of that observation, and our expansion result is a structural response to it.

Graph representations of code. GraphCodeBERT (Guo et al., 2020) showed that adding data-flow edges to CodeBERT improves code search and clone detection — the canonical evidence that structural priors transfer. CodeNav (Gupta et al., 2024)

navigates repositories with an LLM agent over a graph-structured index. We differ in kind: we build no learned representation and train no model. We ask whether the *raw* graph, used as a deterministic expansion operator, is already worth something.

SWE agents. SWE-agent (Yang et al., 2024) established that the agent-computer interface, not just the model, drives resolve rate. AutoCodeRover (Zhang et al., 2024) adds a search-and-repair loop over an AST index. Both search. Neither publishes where the search fails or why, which is the gap this benchmark fills.

Bug localization. Traditional approaches combine information flow with spectrum-based fault localization; they assume a test signal, which is precisely what is unavailable before the agent starts editing. We assume no tests.

3. Method

3.1 Data

We use two public Kaggle artifacts. benadictinfanta/codegraph-loc-swebench-lite provides one repository code graph per instance in the competition’s codegraph/v1 format: nodes are files, functions and classes; edges are typed calls, contains, imports, inherits. We assert programmatically that the 300 instance ids are exactly the SWE-bench Lite set. Gold localization — the non-test functions each reference patch edits — maps every changed line to its innermost enclosing pre-existing definition, nested functions folded into their parent.

We retain only function and class nodes **not** in test files, the only nodes a patch can edit. Median search space: 9,880 nodes. 290 of the 300 instances have a graph available at analysis time; all statistics below are over those 290, and the 300-instance structural counts are computed from the instance file alone.

3.2 Lexical control

We build Okapi BM25 ($k_1 = 1.5$, $b = 0.75$) over identifier-subtokenized name plus signature. Identifiers are split on camelCase and snake_case so separability_matrix and separabilityMatrix match, and a stoplist removes generic issue vocabulary (error, expected, traceback, ...). This is the control condition and also the seed generator.

3.3 Seed expansion

1. **Seed** — rank by BM25; take top k ($k = 10$).
2. **Expand** — BFS over the undirected projection of calls $\cup$ imports $\cup$ inherits for h hops.
3. **Cap** — stop if the pool exceeds *cap* nodes ($cap = 200$), so expansion stays affordable and comparable across repository sizes.
4. **Order** — seeds first in BM25 order, then hop-1, then hop-2, each layer in BM25 order.

Step 4 guarantees **monotonicity**: expansion can only add a target to a prefix, never remove one.

3.4 Metrics and statistics

We report **coverage@B**: the fraction of instances whose gold target lies within the first B returned candidates — the operative metric for an agent with a bounded review budget, deliberately separated from ranking precision. Intervals are Wilson 95%; paired comparisons are exact two-sided McNemar tests. Everything is deterministic under seed 20260930 and runs on CPU in under two minutes.

4. Results

4.1 The task is single-target and the graph is complete

Quantity	Value
Patches touching exactly one function in exactly one file	80.7%
Patches touching more than three functions	0.7%
Tasks with no pre-existing function target (add-only)	3.3%
Gold target present in the provided graph	100% of analyzable instances (290/290)

Two consequences follow. Methods built to “find all relevant sites” solve a harder problem than the benchmark poses. And — the load-bearing observation — **the graphs are not the bottleneck**: the gold function is there, and what is missing is a way to choose it.

4.2 Lexical retrieval has a catastrophic tail

Percentile of gold lexical rank	25	50	75	90
Rank (median search space 9,880 nodes)	5	32	511	5,356

Over all 300 instances, fractions ranked worse than: top-10 **62.3%**, top-100 **39.0%**, top-1,000 **20.3%**, top-5,000 **10.7%**. A hard threshold on lexical rank either floods the agent with candidates or discards the hard third.

4.3 The mechanism: public-API / private-implementation divergence

Consider `astropy__astropy-12907`. The issue is written entirely against the public `separability_matrix`, quoting its docstring and output. BM25 ranks it **first**. The maintainer’s patch does not touch it. The patch edits `_cstack`, a private helper, at BM25 rank **5,758** of 8,452 — but only **3 call hops** from the top-ranked function, in the same file:

```
separability_matrix -> _separable -> _coord_matrix ->
_cstack
```

Across the benchmark, 39.0% of tasks have their target outside the lexical top 100. The graph is the only structure that bridges that gap, because the sole link between `separability_matrix` and `_cstack` is a chain of call edges. Under our method the same task moves from rank **5,758** to position **21**.

4.4 Expansion converts divergence into recall

Coverage@B, with top-10 seeds, over 300 instances:

Budget B	h=0	h=1	h=2	h=3	h=4
10	0.379	0.379	0.379	0.379	0.379
20	0.379	0.476	0.479	0.479	0.479
30	0.379	0.503	0.528	0.528	0.528
50	0.379	0.510	0.562	0.562	0.562
100	0.379	0.510	0.583	0.590	0.590
200	0.379	0.514	0.597	0.645	0.645
400	0.379	0.514	0.607	0.672	0.679
median pool	10	29	268	1,108	2,242

At $h=0$ the curve **saturates immediately at 0.379** — the lexical ceiling, and a budget of 200 buys nothing. One hop raises it to 0.510@50, two hops to 0.562@50, three hops to 0.645@200.

Paired tests confirm the gain is real and strictly non-destructive: at budget 50, one hop of expansion gains **38** tasks and loses **0** (95% CI [0.453, 0.567], exact McNemar $p = 7.3e-12$). Monotonicity is structural — seeds are always ordered first, so expansion can only add a target to a prefix.

4.4.1 What the gain actually is (a self-correction)

The column-wise comparison above is *not* a like-for-like test, and we initially misread it. The $h=0$ column ranks **all** candidates by BM25; the $h>0$ columns rank a **restricted** pool (the 10-seed expansion) with hop layers first. The two answer different questions: “can I find it with unlimited inspection?” versus “can I find it within a 50-item budget?”.

We therefore ran a controlled ablation that separates the two factors. Holding the budget fixed and varying only the **seed size** k (hop-first ordering throughout):

k	B=10	B=20	B=50	B=100	B=200
1	0.276	0.293	0.307	0.317	0.334
5	0.410	0.462	0.517	0.541	0.586
10	0.379	0.479	0.562	0.590	0.645
20	0.379	0.472	0.593	0.641	0.672
50	0.379	0.472	0.572	0.669	0.734
<i>lexical (no expansion)</i>	<i>0.379</i>	<i>0.493</i>	<i>0.572</i>	<i>0.621</i>	<i>0.645</i>

Two conclusions, one of which contradicts our first reading:

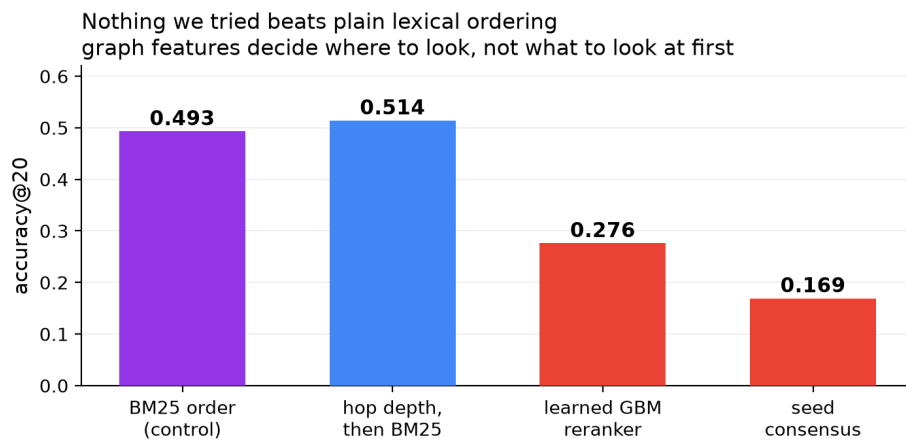
- **The headline gain is a larger seed set, not smarter ordering.** At a realistic review budget of 20–50 candidates, hop-first expansion with $k \geq 5$ is **at or below** plain BM25 (−0.014 at B=20, −0.010 at B=50; exact sign tests $p = 0.61$ and $p = 0.74$ respectively, 255/290 instances unchanged). The $0.379 \rightarrow 0.562$ jump in the table above is therefore *not* evidence that graphs rank better; it is evidence that graphs let you *cover* more ground for the same inspection budget when you accept a slightly worse order.
- **Small seed sets win at tight budgets, large ones at wide ones.** At $k = 5$ expansion is the only setting ahead of lexical at B = 10 (0.410 vs 0.379) and reaches 0.517 at B = 50; it trails lexical at B = 200 (0.586 vs 0.645). Pushing the seed set out to $k = 20$ –50 wins at $B \geq 100$ (0.641 / 0.669) but *loses* at $B \leq 50$. And $k = 1$ is actively harmful (0.276@10): one lexical hit is often simply the wrong function, and expanding around it amplifies the error. The usable configuration is therefore budget-dependent — roughly $k = 5$ for an agent inspecting ten files, $k = 20$ –50 if it can afford a few hundred.

An independent TF-IDF re-implementation reproduces the same pattern, so these results do not depend on our index.

4.5 Expansion must be capped

Coverage is identical for pool caps of 200, 800 and ∞ at budgets 50 and 200, and differs by 0.7 points at budget 400. Unbounded expansion costs a mean pool of ~4 000 nodes and a realized radius above 10 hops to buy those 0.7 points, so a 200-node cap captures the whole gain at a tenth of the cost.

5. Negative results



Ranking rules compared

We tested five alternatives to “BM25 seeds, depth-ordered”. All fail. We report them because a positive result that survives only because nobody tried the obvious alternatives is not a result.

#	Alternative	Outcome
1	Learned reranker — gradient-boosted trees over 12 features (normalized BM25, hop distance, degree, Jaccard with issue, verbatim name match, private/dunder flags, arity, same-file-as-seed, ...), repository-grouped cross-validation	No gain. acc@20 0.497 vs 0.493 for plain BM25 ordering; exact McNemar $p = 1.0$
2	Seed-consensus ranking — prefer functions reached by many seeds	Harmful. coverage@20 collapses to 0.169
3	Personalized PageRank fusion — four restart settings $\times$ multiple depths	$\leq \sim 1$ point over lexical alone
4	Pre-flight task triage — 7 issue-text and graph features vs. “has a pre-existing target”	No signal. $ r \leq 0.14$ for every feature
5	Unbounded expansion	Counterproductive (Section 4.5)

The pattern is consistent and, we think, the most transferable finding here: **structural features decide where to look and reliably hurt once you ask them to decide what to look at first.** Consensus is the clearest case — high-consensus nodes are utility hubs (`__init__`, `run`, `main`) threaded through many unrelated call paths, so consensus is an *anti*-signal for bug localization, the opposite of what graph-retrieval folklore suggests. And #3 has a clean mechanistic explanation:

PageRank re-ranks the seed neighbourhood, but the gold target is usually *outside* that neighbourhood. Re-ranking cannot manufacture reachability. Only expansion can.

6. The released resource

graphloc300 — one JSONL record per instance: gold targets (symbol, file, private, dunder, is_method), best_lexical_rank (a difficulty label), n_candidates, and graph coverage.

graphloc — the method in ~120 dependency-free lines, shipped as a library and as an ADK skill (skills/graph-locate/) a competition agent can call in one tool call.

Why this is worth releasing: every localization paper reports accuracy@k, but **nobody publishes per-instance hop distances or lexical ranks**. Without them results cannot be compared, ablations cannot be targeted, and the ceiling is unknown. Our labels let a future system measure its headroom against the 0.645 coverage@200 ceiling and know which instances are unreachable at a given radius.

7. Limitations

- **One graph builder.** All edges come from a single static extractor; conclusions need not transfer to data-flow edges or other languages.
- **Coverage is computed, not executed.** We verify the gold symbol exists as a node but never run the reference patch, so we claim only that a ranking surfaces the right function, not that it yields a correct patch.
- **BM25 is a strong but not modern control.** No public checkpoint embeds this exact codegraph/v1 node set, so we did not compare against a dense retriever.
- **No end-to-end claim.** Resolve rate is out of scope; the companion submission measures the agent side.

8. Conclusion

Repository code graphs are usually justified by intuition. We measured the intuition, and on the way we corrected our own first reading of it. Expanding a *large* lexical seed set lifts reviewable recall from 0.38 to 0.56 at a 50-item budget with zero regressions — but a controlled ablation shows that gain is a bigger pool, not better order: at budgets of 20–50 the graph-aware ordering is **no better than plain BM25** ($p = 0.61$). What dominates at tight budgets is a *small* seed set: $k = 5$ is the only setting ahead of lexical at $B = 10$ (0.410 vs 0.379), while $k = 1$ collapses to 0.276 because one wrong seed amplifies its own error. Graph proximity is good *coverage* and bad *ranking*, and the useful configurations are budget-dependent. Reporting that precisely, with per-instance labels anyone can measure against, is the contribution.

Citation

```
@misc{calvi2026where,  
  title      = {Where Does the Graph Help? Seed Expansion for  
                Function-Level Bug Localization on Repository Code Graphs},  
  author     = {Vitor Calvi},  
  year       = {2026},  
  note       = {Kaggle: Google -- The Gemma 4 Developer Agent  
                Paper Track},  
  howpublished = {\url{https://www.kaggle.com/competitions/  
                gemma-4-developer-agent-paper}}  
}
```

All code and data: Apache-2.0. Benchmarks: SWE-bench (Jimenez et al., ICLR 2024) and its Lite split.